

Fonctions récursives



Un *algorithme récursif* est un algorithme qui résout un problème en calculant des solutions d'instances plus petites du même problème. Il se réemploie donc lui-même. Cela peut s'illustrer par les images suivantes :



1. Principe de base :

Une fonction récursive simple s'écrit sous la forme :

```
def fonctionRecursive(args)
    if condition arret:
        return valeur
    fonctionRecursive(argument)
```

Pour écrire une fonction récursive, il faut :

- ➔ Déterminer le type de données à renvoyer
- ➔ Écrire la condition d'arrêt (appelé aussi cas de base) pour interdire une récursivité infinie
- ➔ Écrire l'appel récursif

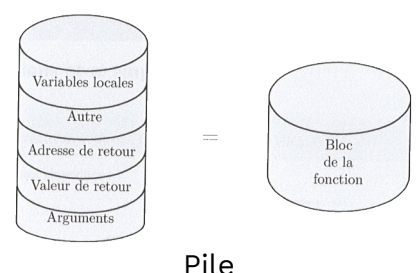
2. Intérêt de la récursivité

La récursivité fournit des algorithmes concis et élégants. Il s'agit à la fois d'un style de programmation mais également d'une technique pour définir des concepts et résoudre certains problèmes qu'il n'est parfois pas facile de traiter en programmant uniquement avec des boucles.

Il faut cependant se méfier de la complexité, de plus il faut bien s'assurer de définir un cas d'arrêt pour interdire une récursivité infinie.

3. Gestion de la mémoire lors d'un appel récursif

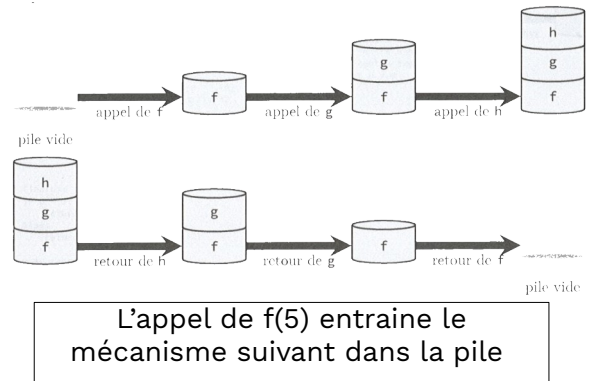
Pour chaque appel de fonction, les données (arguments de la fonction, valeur de retour, adresse de retour, variables locales) sont stockés dans un espace réservé de manière empilée dans la mémoire nommé « la pile ». Cette pile est une structure de données linéaire.



Gestion des appels de fonctions

Considérons l'appel des fonctions suivantes :

```
def h(x):  
    return x+1  
def g(x):  
    return h(x)+2  
def f(x):  
    return g(x)+1
```



Pile d'appel sous Python

Lors de l'appel d'une fonction récursive une pile est utilisée. En Python, la taille de la pile est limitée à 1000 par défaut. En cas de dépassement le message d'erreur suivant s'affiche : `RuntimeError : maximum recursion depth exceeded`

Types de récursivité

Il existe plusieurs types de récursivité :

- ➔ **récursivité simple** : un algorithme récursif est simple ou linéaire si chaque cas se résout en au plus un appel récursif. Le calcul de la factorielle en est un exemple.
- ➔ **récursivité multiple** : un algorithme récursif est multiple si l'un des cas qu'il distingue se résout avec plusieurs appels récursifs. Si "plusieurs=2" on parle de récursivité binaire.
- ➔ **récursivité terminale** : un algorithme est récursif terminal si l'appel récursif est la dernière instruction de la fonction. La valeur retournée est directement obtenue par un appel récursif.

Problème de la terminaison

Pour garantir la terminaison d'une fonction récursive il faut

- ➔ S'assurer qu'on a traité les cas de base qui n'effectuent pas d'appel récursif.
- ➔ Vérifier qu'à chaque appel récursif, les valeurs passées en arguments sont plus "petites" que les précédentes, en terme de taille par exemple

